Machine Learning Experiment Guidebook

Experiment 1：KNN Algorithm for Iris Flower Classification

1. Objective

The objective of this experiment is to introduce the K-Nearest Neighbors (KNN) algorithm and its application in solving the iris flower classification problem. By the end of this experiment, you should be able to understand the concept of KNN, implement the algorithm in Python, and evaluate the model's performance in classifying iris flowers.

2. Environment

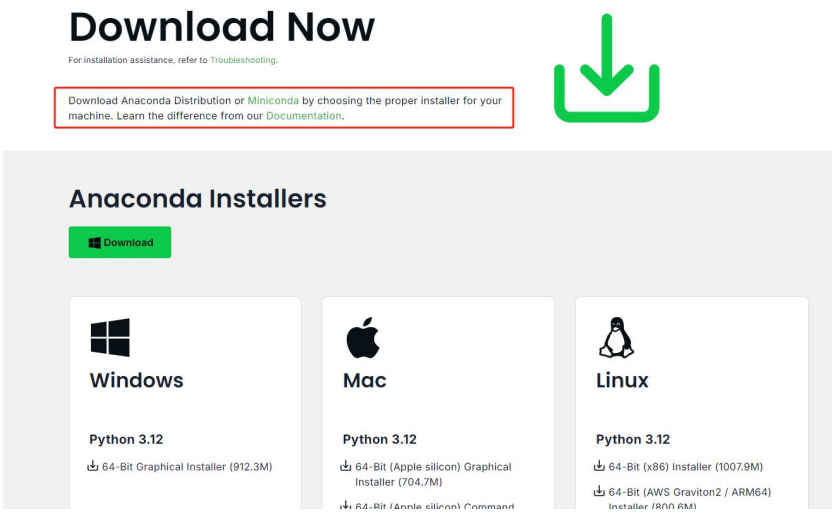
- Operating System: Windows 10
- Python programming environment: Anaconda & Jupyter Notebook

3. Experiment Tasks

3.1 Install Anaconda and Jupyter Notebook

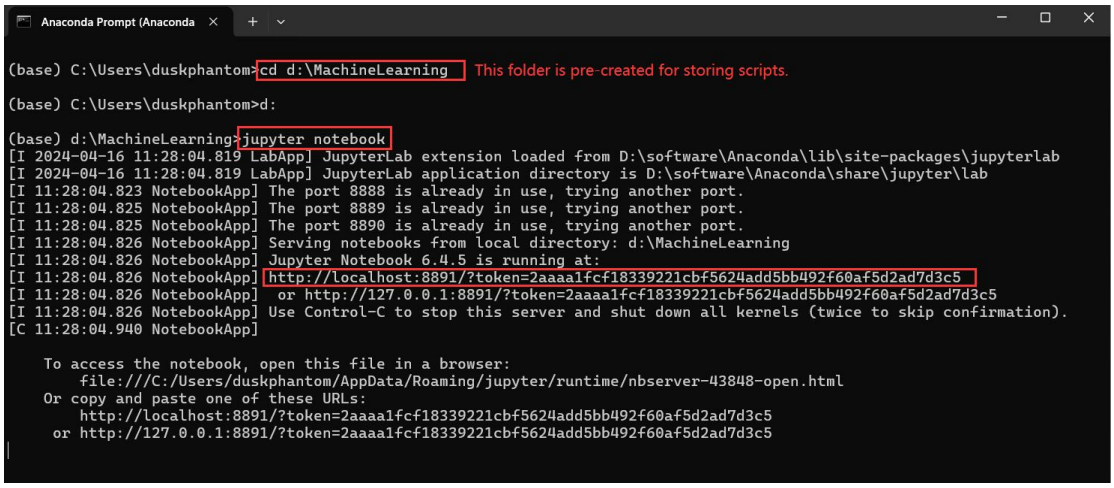
Note: The lab computers have English operating systems. You can select your preferred settings during startup. And they have already been pre-installed with Anaconda.

(1) Go to the official website of Anaconda (https://www.anaconda.com/download) and select the appropriate version for your operating system.

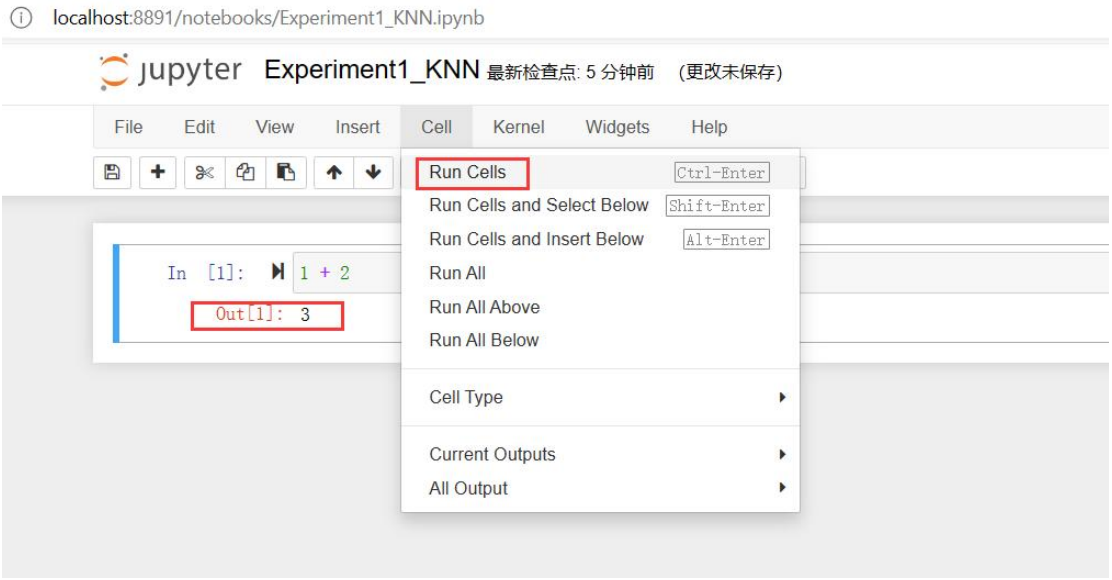


(2) Download and complete the installation.

(3) After the installation is complete, open Anaconda Prompt. Enter the command "jupyter notebook" to start Jupyter Notebook.



(4) A web browser will open, displaying the Jupyter Notebook interface.You can now create, edit, and run code in the Jupyter Notebook environment.



If you are not familiar with Python , you can refer to the following tutorial.
The Python Tutorial — Python 3.9.18 documentation

Note: Please follow the below-listed steps to train a classifier using the KNN algorithm in the Iris dataset, and make sure to save the screenshots of both the experimental code and its output results in your experiment report.

3.2 Load Iris Dataset

The Iris flower data set is a multivariate data set introduced by the British statistician and biologist Ronald Fisher. The data set consists of 50 samples from each of three species of Iris (Iris Setosa, Iris virginica, and Iris versicolor). Four features were measured from each sample: the length and the width of the sepals and petals, in centimeters.

(1) Invoke the load_iris() function to load the data.

```
import numpy as np                  # NumPy library for numerical computations, including correlation calculation
import matplotlib.pyplot as plt     # Matplotlib's plotting module for creating visualizations
```

```
from sklearn import datasets        #Load the Iris dataset from scikit-learn's datasets module
iris = datasets.load_iris()
```

(2) The iris object returned by load_iris is a Bunch object, which is very similar to a dictionary, and it contains keys and values inside.

```
iris.keys()
```

```
dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module'])
```

(3) The value of the DESCR key is a simple explanation of the data set.

```
print(iris.DESCR)
```

```
.. _iris_dataset:

Iris plants dataset
--------------------

**Data Set Characteristics:**

    :Number of Instances: 150 (50 in each of three classes)
    :Number of Attributes: 4 numeric, predictive attributes and the class
    :Attribute Information:
        - sepal length in cm
        - sepal width in cm
        - petal length in cm
        - petal width in cm
        - class:
                - Iris-Setosa
                - Iris-Versicolour
                - Iris-Virginica

    :Summary Statistics:

    ============== ==== ==== ======= ===== ====================
                    Min  Max   Mean    SD   Class Correlation
    ============== ==== ==== ======= ===== ====================
    sepal length:   4.3  7.9   5.84   0.83    0.7826
    sepal width:    2.0  4.4   3.05   0.43   -0.4194
    petal length:   1.0  6.9   3.76   1.76    0.9490  (high!)
    petal width:    0.1  2.5   1.20   0.76    0.9565  (high!)
    ============== ==== ==== ======= ===== ====================

    :Missing Attribute Values: None
    :Class Distribution: 33.3% for each of 3 classes.
    :Creator: R.A. Fisher
    :Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)
    :Date: July, 1988

The famous Iris database, first used by Sir R.A. Fisher. The dataset is taken
from Fisher's paper. Note that it's the same as in R, but not as in the UCI
Machine Learning Repository, which has two wrong data points.
```

(4) The value of the feature_names key is a list of strings that explains each feature.

```
iris.feature_names
```

```
['sepal length (cm)',
 'sepal width (cm)',
 'petal length (cm)',
 'petal width (cm)']
```

(5) The value of the target_names key is a string array, containing the species of Iris flowers that we are to predict.

```
iris.target_names
```

```
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

(6) The data is contained in the target and data fields. Inside the data are the sepal length, sepal width, petal length and petal width. The data is in the form of a NumPy array, and each row of the data array corresponds to a flower, and the columns represent the four measurement data of each flower.

```
iris.data[:5] #show the first 5 rows from the dataset
```

```
array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
       [5. , 3.6, 1.4, 0.2]])
```

```
iris.data.shape
```

```
(150, 4)
```

(7) The target array contains the species of each measured flower, which is also a NumPy array, and the species are converted into integers ranging from 0 to 2: 0 stands for setosa, 1 stands for versicolor, and 2 stands for virginica.

```
iris.target
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

```
iris.target.shape
```

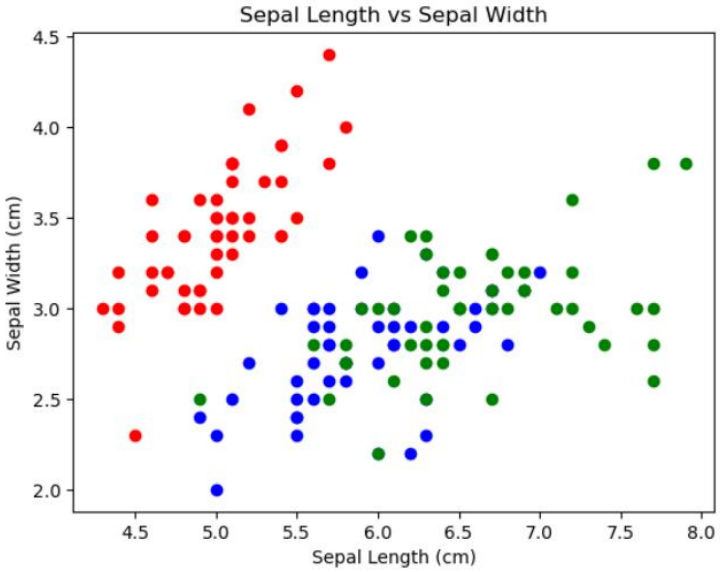
```
(150,)
```

3.3 Data analysis

(1) The bellow graph shows relationship between the sepal length and width.

```
X = iris.data[:,:2]      # Take out the information of the sepal length and width.
y = iris.target
plt.scatter(X[y==0,0],X[y==0,1],color='red')
plt.scatter(X[y==1,0],X[y==1,1],color='blue')
plt.scatter(X[y==2,0],X[y==2,1],color='green')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.title('Sepal Length vs Sepal Width')
plt.show()
```

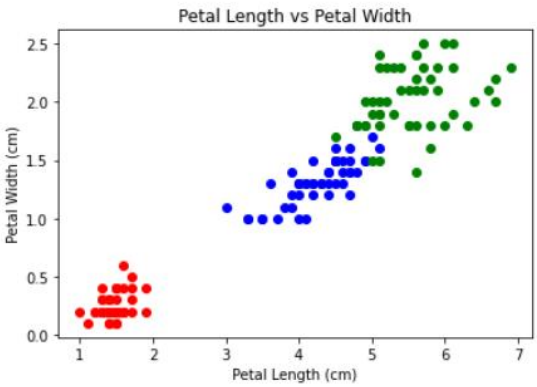
```
plt.scatter(X[y==0,0],X[y==0,1],color='red')
plt.scatter(X[y==1,0],X[y==1,1],color='blue')
plt.scatter(X[y==2,0],X[y==2,1],color='green')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.title('Sepal Length vs Sepal Width')
plt.show()
```



(2) Now we will check relationship between the petal length and width.

```
X = iris.data[:,2:]
plt.scatter(X[y==0,0],X[y==0,1],color='red')
plt.scatter(X[y==1,0],X[y==1,1],color='blue')
plt.scatter(X[y==2,0],X[y==2,1],color='green')
plt.xlabel('Petal Length (cm)')
plt.ylabel('Petal Width (cm)')
plt.title('Petal Length vs Petal Width')
plt.show()
```

```
plt.scatter(X[y==0,0],X[y==0,1],color='red')
plt.scatter(X[y==1,0],X[y==1,1],color='blue')
plt.scatter(X[y==2,0],X[y==2,1],color='green')
plt.xlabel('Petal Length (cm)')
plt.ylabel('Petal Width (cm)')
plt.title('Petal Length vs Petal Width')
plt.show()
```



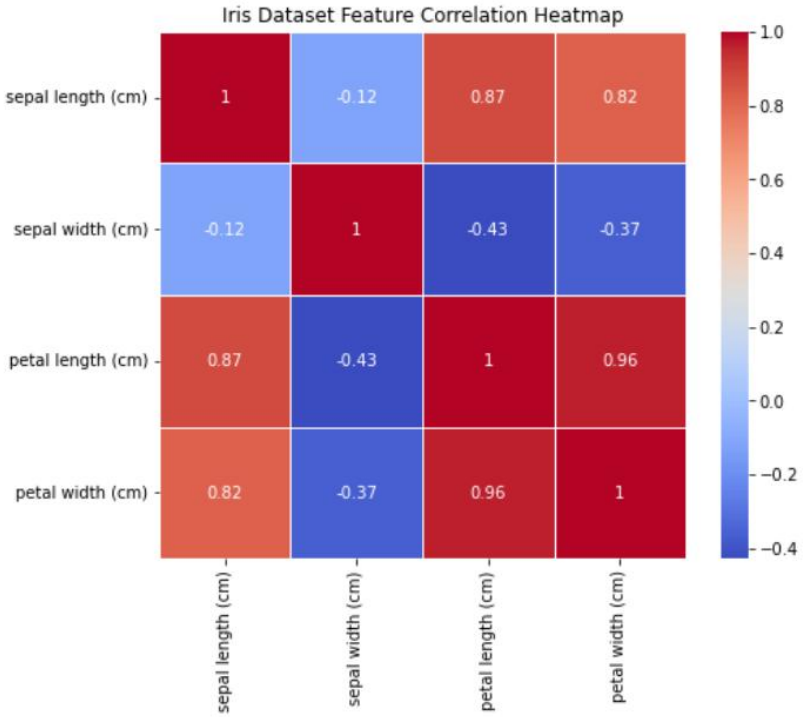
As we can see that the Petal Features are giving a better cluster division compared to the Sepal features. This is an indication that the Petals can help in better and accurate Predictions over the Sepal. We will check that later.

(3) The heatmap allows for easy interpretation of which features are positively or negatively correlated with one another in the dataset.

When we train any algorithm, the number of features and their correlation plays an important role. If there are features and many of the features are highly correlated, then training an algorithm with all the features will reduce the accuracy. Thus features selection should be done carefully. This dataset has less features but still we will see the correlation.

```
X = iris.data
pip install seaborn # Seaborn, a data visualization library based on Matplotlib
import seaborn as sns

correlation_matrix = np.corrcoef(iris.data, rowvar=False)
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=.5, square=True,
                xticklabels=iris.feature_names, yticklabels=iris.feature_names)
plt.title('Iris Dataset Feature Correlation Heatmap')
plt.show()
```

```
import seaborn as sns  # Seaborn, a data visualization library based on Matplotlib that provides enhanced styles and functions

# Calculate the Pearson correlation coefficient matrix for the features
correlation_matrix = np.corrcoef(iris.data, rowvar=False)
# Set rowvar=False because each row represents an observation (sample)

# Set up a new figure with dimensions of 8 inches by 6 inches
plt.figure(figsize=(8, 6))

# Generate a heatmap using Seaborn to visualize the correlation matrix
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=.5, square=True,
           xticklabels=iris.feature_names, yticklabels=iris.feature_names)
# annot=True displays the correlation values within the cells
# cmap sets the color map to 'coolwarm' for a divergent color scheme
# linewidths controls the width of the border lines around each cell in the heatmap

# Add a title, then display the plotted heatmap
plt.title('Iris Dataset Feature Correlation Heatmap')
plt.show()
```



If you have the issue that the values is missing in the heatmap, please update your matplotlib version with the following command :

#pip install --upgrade seaborn matplotlib pandas

Pip install --user matplotlib==3.7.3

Observation: What does the heatmap show?

3.4 Model Training

The given problem is a classification problem. We will be using the K-Nearest Neighbors (KNN) classification algorithms to build a model.

Before we start, we need to clear some Machine Learning (ML) notations.

Classification: samples belong to two or more classes and we want to learn from already labeled data how to predict the class of unlabeled data

Attributes: An attribute is a property of an instance that may be used to determine its classification. In the Iris dataset, the attributes are the petal and sepal length and width. It is also known as Features.

Target variable: in the machine learning context is the variable that is or should be the output. Here the target variables are the 3 flower species.

Steps To Be followed When Applying an Algorithm:

1. Split the dataset into training and testing dataset. The testing dataset is generally smaller than training one as it will help in training the model better.
2. Initialize and the KNN model with “n_neighbors=k”.
3. Then pass the training dataset to KNN algorithm to train it. We use the .fit() method.
4. Then pass the testing data to the trained algorithm to predict the outcome. We use the .predict() method.
5. We then check the accuracy by passing the predicted outcome and the actual output to the model.

✓ Splitting The Data into Training And Testing Dataset:

```
from sklearn.model_selection import train_test_split  # For splitting the dataset into training and testing sets

# Split the dataset into features (X) and target labels (y)
X = iris.data  # Features
y = iris.target  # Target labels

# Define the desired test set size or the train-test split ratio
test_size = 0.3  # 30% of the dataset will be used for testing
# Alternatively, you can specify the train_test_split_ratio directly:
# train_test_split_ratio = 0.7  # 70% of the dataset will be used for training

# Use train_test_split function to divide the dataset into training and testing sets
# Shuffle the data before splitting if desired (default is True)
#fixed random state for reproducibility
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size, random_state=48)

# Print the sizes of the training and testing sets
print(f"Training set size: {len(X_train)} samples")
print(f"Test set size: {len(X_test)} samples")
```

```
Training set size: 105 samples
Test set size: 45 samples
```

✓ Initialize and train the KNN classifier model:

```
from sklearn.neighbors import KNeighborsClassifier  # K-Nearest Neighbors (KNN) classifier implementation

# Instantiate a KNN classifier with k=3 (considering 3 nearest neighbors for classification)
knn = KNeighborsClassifier(n_neighbors=3)

# Train the KNN classifier on the training data
knn.fit(X_train, y_train)
```

```
KNeighborsClassifier(n_neighbors=3)
```

✓ Make predictions on the testing data

```
# Make predictions on the testing data using the trained KNN classifier
y_pred = knn.predict(X_test)
y_pred
```

```
array([1, 1, 2, 0, 1, 2, 0, 1, 0, 1, 2, 0, 0, 2, 1, 1, 0, 1, 2, 2, 0, 2,
       1, 1, 2, 0, 0, 2, 2, 1, 2, 1, 2, 0, 1, 2, 2, 1, 0, 1, 1, 1, 2, 2,
       1])
```

✓ Evaluate the model's accuracy:

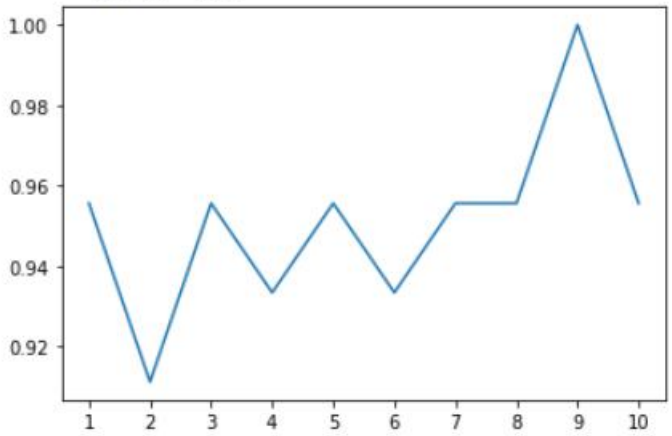
```
from sklearn.metrics import accuracy_score  # For evaluating model performance

# Print the overall accuracy score (fraction of correctly classified instances)
print("\nAccuracy Score:", accuracy_score(y_test, y_pred))
```

```
Accuracy Score: 0.9555555555555556
```

✓ Let's check the accuracy for various values of n for K-Nearest nerighbours.

```
# Define the range of K values
k_values = np.arange(1, 11)

# Initialize a NumPy array to store the accuracy scores
accuracy_scores = np.zeros(k_values.shape)

# Iterate over K values
for i, k in enumerate(k_values):
    # Create a K-Nearest Neighbors classifier with the current K value
    knn = KNeighborsClassifier(n_neighbors=k)

    # Train the model on the training data
    knn.fit(X_train, y_train)

    # Make predictions on the test set
    predictions = knn.predict(X_test)

    # Compute and store the accuracy score corresponding to the current K value
    accuracy_scores[i] = accuracy_score(predictions, y_test)

    print("k = ",k,"  Accuracy Score:", accuracy_scores[i])

# Plot the relationship between accuracy and K values using matplotlib
plt.plot(k_values, accuracy_scores)
plt.xticks(np.arange(1, 11))
```

```
k =  1   Accuracy Score: 0.9555555555555556
k =  2   Accuracy Score: 0.9111111111111111
k =  3   Accuracy Score: 0.9555555555555556
k =  4   Accuracy Score: 0.9333333333333333
k =  5   Accuracy Score: 0.9555555555555556
k =  6   Accuracy Score: 0.9333333333333333
k =  7   Accuracy Score: 0.9555555555555556
k =  8   Accuracy Score: 0.9555555555555556
k =  9   Accuracy Score: 1.0
k =  10   Accuracy Score: 0.9555555555555556
```



Above is the graph showing the accuracy for the KNN models using different values of k.

We used all the features of iris in above models. Now we will use Petals and Sepals separately.

✓ Creating Petals And Sepals Training Data

```
X_Sepal = iris.data[:,:2] # Take out the information of the sepal length and width.

X_Petal = iris.data[:,2:] # Take out the information of the Petal length and width.

y = iris.target  # Target labels

test_size = 0.3 # 30% of the dataset will be used for testing

X_train_Sepal, X_test_Sepal, y_train_Sepal, y_test_Sepal = train_test_split(X_Sepal, y, test_size=test_size, random_state=0) #sepal

X_train_Petal, X_test_Petal, y_train_Petal, y_test_Petal = train_test_split(X_Petal, y, test_size=test_size, random_state=0) #petals
```

✓ K-Nearest Neighbours

```
knn_Sepal = KNeighborsClassifier(n_neighbors=3)

knn_Sepal.fit(X_train_Sepal,y_train_Sepal)

prediction = knn_Sepal.predict(X_test_Sepal)

print('The accuracy of the KNN using Sepal is:',accuracy_score(prediction,y_test_Sepal))

knn_Petal = KNeighborsClassifier(n_neighbors=3)

knn_Petal.fit(X_train_Petal,y_train_Petal)

prediction = knn_Petal.predict(X_test_Petal)

print('The accuracy of the KNN using Petel is:',accuracy_score(prediction,y_test_Petal))
```

```
The accuracy of the KNN using Sepal is: 0.7333333333333333
The accuracy of the KNN using Petel is: 0.9777777777777777
```

Observations: What do you discover from the results?